

Contents

The Rulith Decision Kernel — Proof-Carrying Decisions for Autonomous Agents	1
Self-driving agents that cannot overreach their commitments.	1
Executive summary	1
1. The problem: capability is outrunning controllability	2
2. The idea: the decision must be <i>earned</i> , not <i>asserted</i>	2
3. The kernel: one tick	2
4. Six honest states — no fake “done”	3
5. Touching the world takes two beats	3
6. What “can’t go rogue” precisely means: the twelve invariants	3
7. The honest floor: the kernel marks what it cannot judge	4
8. Why it is model-independent — and certifiable	4
9. From a task to a life: the five objects	5
10. Where this fits — and an honest scope	5
11. The next rung: from a life to a fleet	6
12. Conclusion	6
Acknowledgments	6

The Rulith Decision Kernel — Proof-Carrying Decisions for Autonomous Agents

Self-driving agents that cannot overreach their commitments.

Victor Shaw · Independent Researcher · michaltina@hotmail.com

White paper · release draft, 2026-06. Companion to the empirical paper *The Driving Floor* and the open-source **rulith** reasoning board. This document states the theory and the guarantees; the mechanism is implemented and demonstrated, and the honest gap to large-scale real-world deployment is stated in §10.

Executive summary

Autonomous agents and robots are crossing the line from *suggesting* actions to *taking* them — placing trades, moving machines, changing records, calling tools — without a human in every loop. The capability to do this is racing ahead. The ability to *trust* it is not.

Today’s autonomy is often built from black-box learned policies with safety bolted on as a post-hoc filter. That can produce behaviour that is *usually* fine, but it does not give a checkable reason why a costly action was allowed. For anything where a wrong action carries real cost — a robot near a person, an agent with money or infrastructure access — “usually fine” is not enough.

The Rulith decision kernel is a different shape: **propose-adjudicate**. The learned model *proposes* what to do; the kernel *adjudicates* whether it may. An action is dispatched only if it is **derived** (earned by deductive closure, not asserted), **authorized** (within commitments the agent cannot relax on its own), and **grounded** (traceable to evidence on a named channel). Whatever the kernel *cannot* decide — the truth of a premise, whether progress is possible, the state of the outside world — it **marks honestly** rather than faking a verdict.

The result is a precise, unified guarantee — **twelve invariants** (I1–I12, plus four cross-session memory invariants, I13–I16) — that holds **regardless of the model driving it**. The agent cannot overreach without trace, complete without evidence, spin without bound, or quietly move its own goalposts. Because the guarantee is structural (deductive gates + provenance) rather than behavioural, it is **model-independent** and, crucially, **certifiable**: every decision carries an audit

trail of *why*. That is the missing layer: not a smarter model, but a decision surface that can be inspected, constrained, and trusted.

1. The problem: capability is outrunning controllability

It is widely agreed that building deterministic safety around non-deterministic AI is one of the hardest engineering problems of the decade — in software a bug is a crash; in an embodied or financially-empowered agent, a bug is a broken bone or an unauthorized transaction. The prevailing answers are:

- **End-to-end learned policies** (vision-language-action models, agent frameworks). Excellent at generalization and dexterity; fundamentally **black boxes** whose decisions cannot be guaranteed or audited.
- **Bolt-on guardrails / runtime monitors** (rule checkers, temporal-logic constraints, verified policy filters). Useful, but they sit *outside* a black-box policy and **veto** specific bad actions. They check single properties; they do not make the decision itself *earned*, and they do not give a coherent, auditable account of *why* the agent did what it did.

Neither gives the one thing autonomy actually requires: a decision you can **trust without a human watching**, with a **record that proves it was sound**.

2. The idea: the decision must be *earned*, not *asserted*

The Rulith decision kernel inverts the usual arrangement. Instead of “the model decides and a checker vetoes,” it is **propose-adjudicate**:

- The **model proposes** — it models the situation as facts and rules, and proposes the next operations, actions, and evidence.
- The **kernel adjudicates** — it runs deductive closure, maintains consistency, computes exactly, and **gates every conclusion**. A guarded conclusion (e.g. “this action is permitted,” “this task is done”) is admitted **only if the closure derives it** from grounded premises. A bare assertion never passes.

So safety is not a filter laid over a black box; it is that **the decision itself only exists if it was deductively earned**. The model supplies intuition and proposals; the kernel supplies the exact, deductive half. This is the same propose-adjudicate substrate as the open **rulith** reasoning board. The Rulith decision kernel applies the board’s adjudication, tick by tick, to a system that acts on the world.

3. The kernel: one tick

A self-driving life is a loop. Each tick:

observe

- ingest typed observations (onto a trusted channel)
- derive the board's closure
- expose the frontier (what may be done next)
- the model proposes ops / actions / evidence
- working memory adjudicates (derivation + consistency gates)
- the authority gate decides what is dispatchable
- the host executes only dispatchable actions
- effect_confirmed / effect_failed returns through the trusted channel
- the receipt updates: open / ready / closed / stale / blocked

The division of labour is sharp and is the whole safety story:

**The model proposes. The kernel adjudicates. The host mechanically executes.
The human owns goals, constraints, permissions, and final policy.**

No party can do another's job: the model cannot grant itself authority or confirm its own effects; the host cannot reason; the kernel cannot want anything. Each is checked by the others.

4. Six honest states — no fake “done”

At every tick the kernel reports exactly one state, computed from three **frontiers** that never impersonate one another (what the *agent* may propose, what the *host* must execute, what a *human* must decide):

state	meaning
driving	the agent still has a next step to propose
actuating	the agent is done proposing; the host owes a real-world effect — this is not done
parked	nothing left but a human decision (authorization / choice / missing input)
done	all three frontiers empty and the receipt is closed
stuck	frontiers empty but the receipt will not close — a real gap; escalate
halted	a trusted halt was requested; overrides everything

The point is honesty: a task with an unconfirmed real-world effect is actuating, **not** done; a task that cannot close is stuck, not silently abandoned. The kernel is never allowed to *look* finished when it is not.

5. Touching the world takes two beats

A real side-effect is never a single step. The host must:

```
dispatchable_action(A)
  → write dispatched(A) first
  → fire the adapter
  → the adapter returns effect_confirmed(A) or effect_failed(A)
```

with hard rules: the **agent cannot write a trusted** dispatched **or** effect_confirmed; a dispatched-but-unconfirmed action **cannot fake a closed receipt**; a re-run **cannot re-fire** an already-dispatched action; and a **failure must enter the board as a fact** that triggers diagnosis and bounded recovery — it is never swallowed by a log. This is how “fail closed, never open” holds against a messy real world.

6. What “can’t go rogue” precisely means: the twelve invariants

“Trustworthy” is not a slogan here; it is a closed set of properties the kernel structurally enforces.

Static (commitment) invariants

1. **No self-certification** — the agent cannot declare itself finished.
2. **No self-grant** — the agent cannot authorize itself.
3. **No trust laundering** — a low-trust premise cannot be relabelled into a high-trust conclusion.
4. **No silent drift** — any change to evidence, permission, or goal makes the receipt go stale.
5. **Trusted-channel monopoly** — trusted facts may enter only through a trusted channel.

6. **Revisable-with-record** — every retraction and revision is traceable.
7. **Decidable exact-or-fail** — on its decidable fragment the kernel terminates and is exact, or fails loudly; it never returns a quietly-wrong “exact” answer.

Dynamic (runtime) invariants

8. **No self-confirmation** — an action’s effect cannot be confirmed by the agent.
9. **At-most-once dispatch** — dispatch is recorded; nothing is blindly replayed.
10. **Quiescence honesty** — a stop must be one of done / parked / stuck / halted.
11. **Bounded recovery** — failure recovery has a budget; exceeding it escalates to a human.
12. **No goalpost-moving** — the agent cannot complete a task by quietly weakening the acceptance criteria. A *committed baseline* (acceptance, constraints, goal structure, and the current phase) is pinned; the agent’s attempts to relax it are blocked unless an authorized amendment is consumed.

(Four further invariants, **I13-I16**, extend the same discipline to cross-session **memory**: persisted facts cannot upgrade their trust tier, carry staleness, keep their provenance, and are only changed through the same adjudicated, recorded entry.)

These twelve invariants define trustworthy self-driving — not a guarantee that the agent always does the right thing, but a guarantee that it *cannot over-reach without trace, complete without evidence, spin without bound, or quietly change the bar it is judged against.* The model is free to be wrong; it is not free to *hide* it.

7. The honest floor: the kernel marks what it cannot judge

A trustworthy system must be honest about the limits of its own knowledge. The kernel guarantees that *reasoning* from premises to conclusions is faithful and exact; it does **not** pretend to know things it cannot. Whatever lies outside what it can decide — whether a ground premise is *true*, whether the frontier of options is *complete*, whether progress is *possible*, the value of an intangible accumulation, the real state of the outside world — the kernel **marks a floor** (a labelled level of trust) rather than faking a verdict.

Trustworthiness here is not “the kernel guarantees the life succeeds.” It is: **at every step, what can be proven is proven, and what cannot be judged is honestly marked.** Marking the limit of the guarantee *is part of* the guarantee.

This is precisely what bolt-on guardrails do not give you: not just a veto on bad actions, but a coherent, labelled account of exactly how much trust each conclusion has earned — which is what an auditor, a regulator, or an operator actually needs.

8. Why it is model-independent — and certifiable

Because the guarantees come from the kernel’s **structure** (deductive gates + provenance), not from the model’s good behaviour, they hold for **any** driving model — strong or weak, cloud or cheap-local. The companion paper *The Driving Floor* shows this empirically on the underlying board: across many models and several task families, **no model could make the board certify an unmade fix (false-certify = 0%)**, and a model whose driving collapses produces an *empty* result, never a *fabricated* one — it **fails closed**.

Two consequences follow:

- **Trust is decoupled from capability.** You do not need a frontier model to get the guarantee; you need a model that can *drive* the kernel. (The *driving floor* — a model’s ability to operate the symbolic interface — is the one model property that matters, and the paper characterizes it.)
- **It is certifiable.** Every conclusion the kernel admits carries a derivation trace to its grounded premises. That trace is exactly the evidence a safety case needs — a structural

audit trail that a black-box policy simply cannot produce. The kernel turns “trust us” into “here is the derivation.”

9. From a task to a life: the five objects

The same machine that drives one task to completion scales to a whole operating *life*. A terminating task is a goal decomposed to verifiable leaves; a non-terminating life is an open *direction* pursued through an ordered sequence of tasks. The kernel handles the full lifecycle through five theoretical objects:

1. **Two-layer policy.** An **immutable constitution** (no-lie / safety / identity) the agent *cannot change* — its enforcement is invariant 12 — plus **mutable phase goals** it pursues one at a time, switching only by an authorized, constitution-checked amendment. A “non-terminating life” is thus a *finite sequence of phases under a constant constitution* — and at every instant what the board must adjudicate (the current phase + constitutional consistency) is decidable.
2. **Accumulation across the decidability spectrum.** Tangible, re-verifiable assets (code, files, tools) ratchet up as verified; intangible ones the kernel cannot re-derive (reputation, skill, trust, real external change) are tracked but capped at an honest value-floor — **never laundered into verified**.
3. **Concurrency as a scheduling forest.** One body, many goals: resources are arbitrated by consume/produce mutual exclusion, scheduling is itself propose-then-adjudicate, and a constitutional (safety) constraint can preempt at any tick.
4. **Perception decoupling.** A fast reflex/perception layer runs *exogenously and in real time*; its only interface to the kernel is dropping facts onto the observation channel, absorbed atomically at tick boundaries. **The kernel is the deliberation layer (tick scale), not a real-time motion controller** — a deliberate separation, with a freshness gate that blocks a major physical action until current perception is ingested.
5. **Driving is exogenous.** The kernel can *measure* progress, surface stalls, and escalate — it does not *guarantee* forward motion. Liveness is honestly measured, not promised.

The thread through all five is the same discipline as §7: whatever the board cannot judge, it marks rather than fakes.

10. Where this fits — and an honest scope

It complements the capability layer; it is not a competitor to it. The Rulith decision kernel is not a vision-language-action model, not a motion controller, and not a “robot OS.” It sits *inside* or *above* any agent or robot stack as the **decision/safety kernel** — the layer that decides *whether* an action is sound and authorized, while the learned policy decides *how* to move and a classical reflex layer keeps the body physically safe. The three compose. The Rulith decision kernel is the part that makes the *decision* trustworthy and auditable.

It differs from runtime-monitor / formal-guardrail peers not by “using symbols” — many do — but by being a **coherent kernel**: the decision must be *derived* (not merely checked after the fact), a **single unified set of invariants** rather than one bespoke property, and **native provenance** on every conclusion that makes the whole thing certifiable. It also *supplies* exact reasoning, not only safety.

Honest scope. What is proven is the *mechanism*: the gates hold, the invariants are enforced (each is pinned to a test), and the guarantees are demonstrated end-to-end in a sandbox and on single real-machine runs, with the underlying board’s model-independence shown empirically in *The Driving Floor*. What remains **empirical and ongoing** is large-scale real-world coverage: robustness against messy real adapters, failures, concurrency and long horizons; the reliability of driving on real tasks; and the correctness of the *domain model* a deployment supplies (the kernel guarantees sound reasoning *from* premises, not the truth of the premises or the completeness of a hand-authored rule set). We treat real-task coverage as a question to be answered by deployment with partners, not assumed from the theory — the same discipline as the paper.

11. The next rung: from a life to a fleet

The same ladder has one rung above a life: a *fleet* — many agents under one shared constitution. **Federation governance** applies propose-adjudicate at population scale. It adds four group-level objects that no single life has: no single agent can change the group's law (amendment is group-governed, never individual); one agent's commitments cannot contradict another's; a verified capability — a domain pack, a proof — propagates across the fleet *carrying its trust floor* (a commons, without laundering); and the population invariant is that *no agent in the group can overreach the modeled law without trace*.

This is rulith's endpoint as **infrastructure**: a shared, governed substrate hosting each agent's board, memory, and rule-packs — closer to a proof-and-capability bank than to a single application. The three open-core seams it needs — per-agent isolation, a capability commons, and shared solvers — are already built. The governance *runtime* (shared-constitution amendment, cross-agent consistency, population-level safety) remains a **blueprint**, theory-first and not yet validated. Claim-level adjudication stays the spine: a fleet is the self-driving loop *federated*, not a new kernel. The honesty is unchanged: *proof-carrying conduct within the modeled law*, not a claim that the law itself is correct.

12. Conclusion

Autonomy will be adopted exactly as fast as it can be *trusted*, and trust requires verifiability. The Rulith decision kernel makes that trust **structural** (a decision must be earned), **unified** (twelve invariants plus four memory invariants define the boundary), **model-independent** (the guarantee does not depend on the driver being wise), and **certifiable** (every admitted decision carries its proof). The reasoning board beneath it is open and paper-backed; the kernel's theory and reference implementation are open for scrutiny at <https://github.com/rulith-dev/rulith>. For a system whose value is *trust*, being verifiable in the open is not a giveaway. It is the product.

We invite researchers, builders, and auditors to break it, extend it, and hold it to its invariants.

Acknowledgments

This white paper was prepared with the assistance of an AI assistant (Claude). The system (rulith), the kernel's theory and reference implementation, and all claims are the author's own.

Companion materials: the empirical paper The Driving Floor (board lift, the driving floor, model-independent trust); the open-source rulith reasoning board (propose-adjudicate substrate, reproducible probes). Author / contact: Victor Shaw · Independent Researcher · michaltina@hotmail.com.